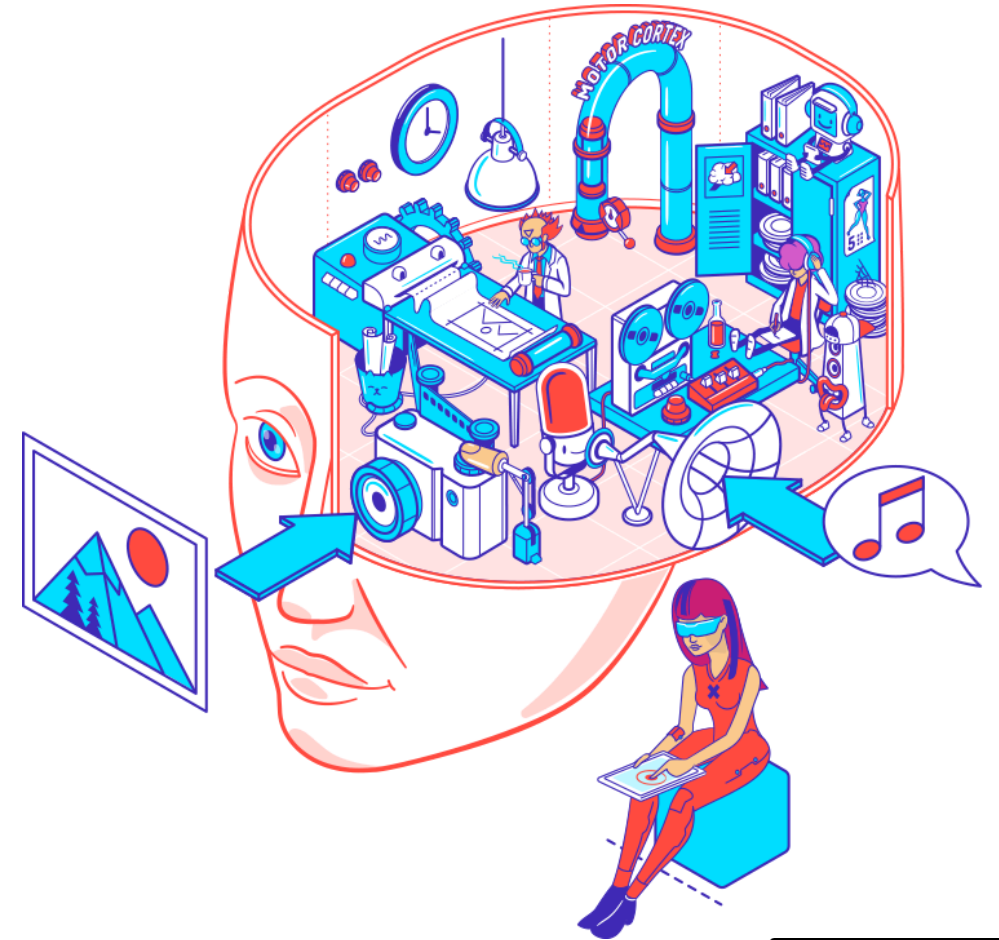


MEDIA PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GENERATION

Prof. Dr. Eike Langbehn

Department of Media Technology
Faculty of Design, Media and Information
Hamburg University of Applied Sciences



AGENDA:

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

INTRO

1. Introduction
2. From Java to Python
3. Raster Graphics
4. **Algorithmic Graphics Generation**
5. Vector Graphics
6. Interactive Graphics
7. Animations

LEARNING OUTCOME

GRAPHICS PROGRAMMING

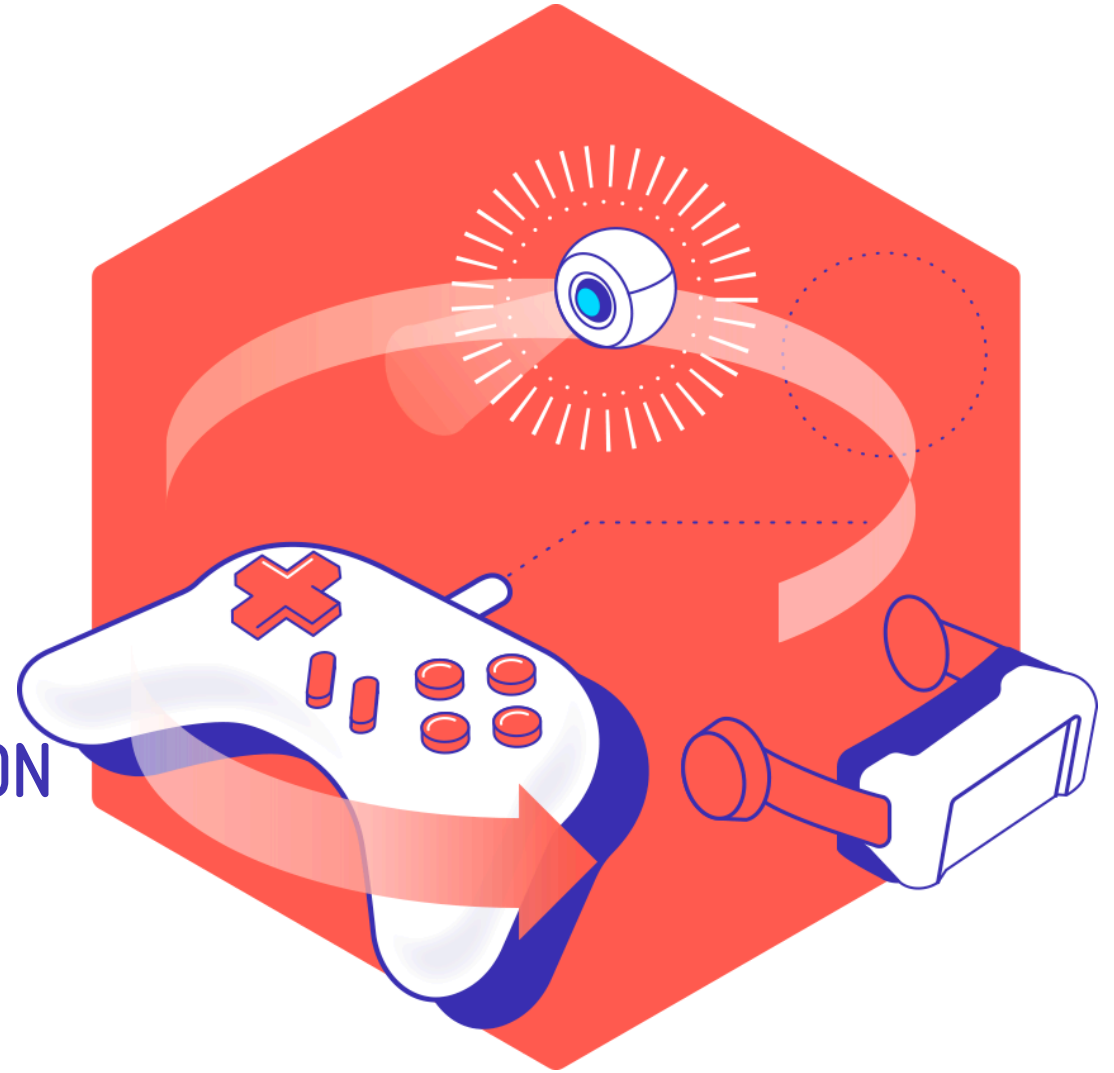
UNIT 4: ALGORITHMIC GRAPHICS GEN.

INTRO

- Understanding how images can be **generated** entirely by **algorithms**
- Knowledge that images are **computed data structures**
- Understanding of how **complex visual structures** can arise from **simple rules**

BASICS

UNIT 4: ALGORITHMIC GRAPHICS GENERATION



PROCESSING VS GENERATION

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

BASICS

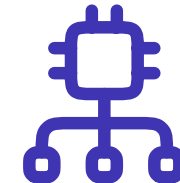
IMAGE PROCESSING

Works on existing digital data
(e.g., modifying a photo)



IMAGE GENERATION

Computational creation
via rules & algorithms



APPLICATION DOMAINS

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

BASICS

Game Environments

Procedural terrain, textures, and foliage.



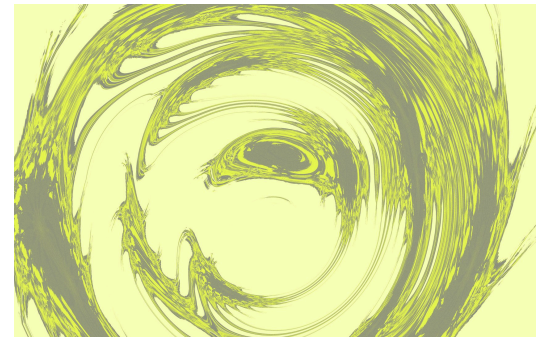
Fractals & Math

Recursive patterns and complex mathematical visualizations.



Generative Art

Abstract compositions driven by algorithms and randomness.



Fluid Simulation

Dynamic simulation of fire, water, and smoke.

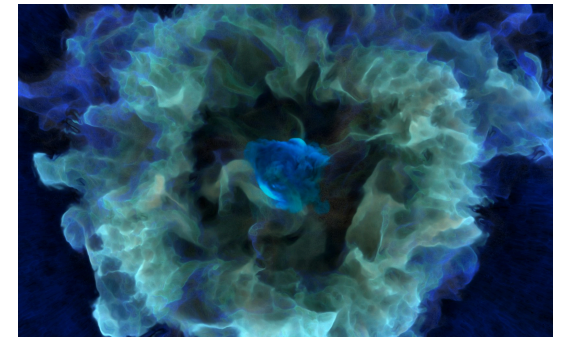


IMAGE = FUNCTION

A function assigns a color value to each position.

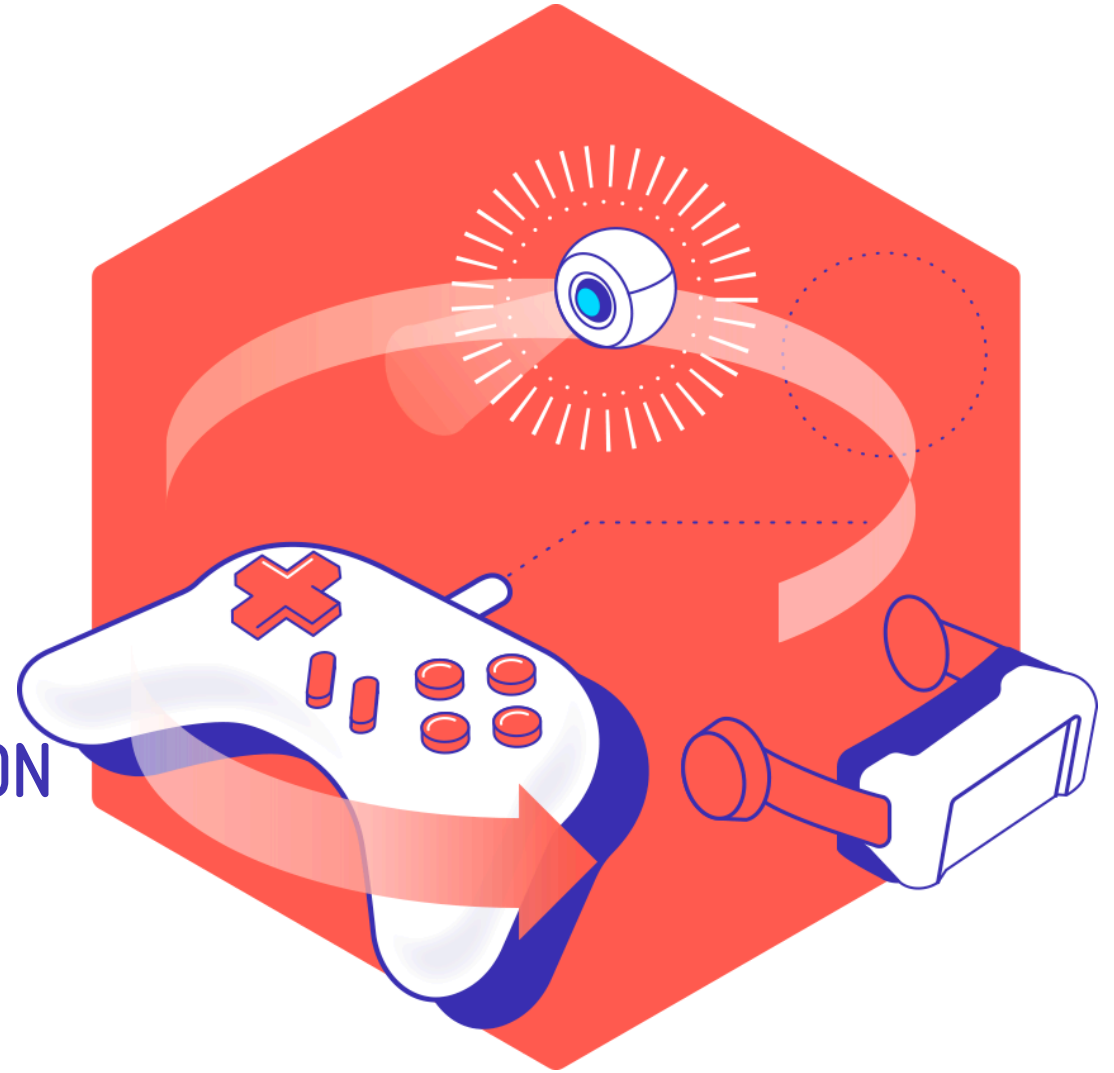
This is the fundamental principle of almost all algorithmic graphics.

$$I(x, y) \rightarrow \text{Color}$$

```
1 def get_pixel_color(x, y):  
2     # Logic to determine color  
3     return (r, g, b)
```

IMAGE SYNTHESIS

UNIT 4: ALGORITHMIC GRAPHICS GENERATION



EXAMPLES

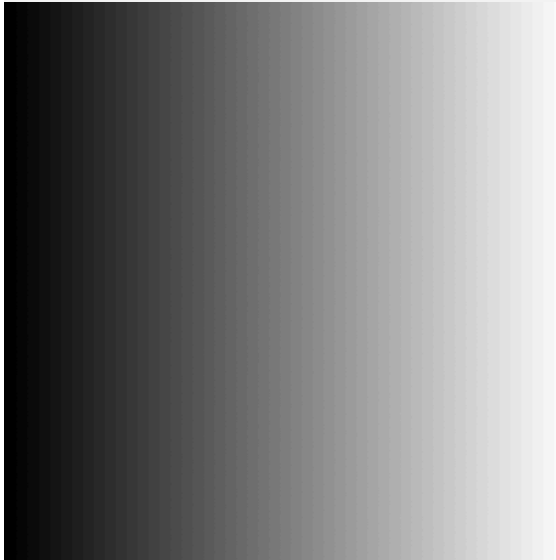
GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

IMAGE SYNTHESIS

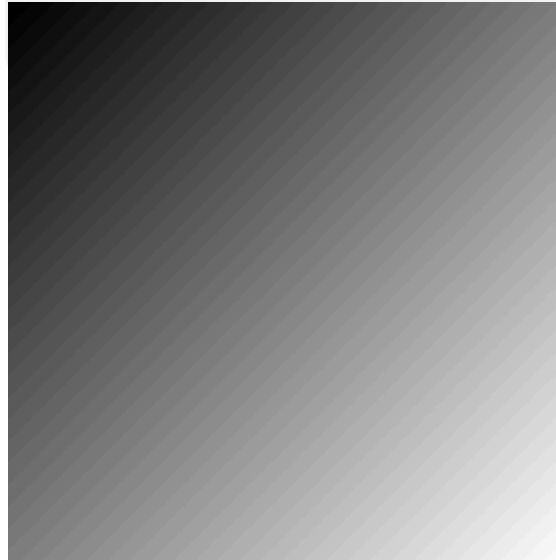
$$I(x, y) = x$$

Horizontal Gradient



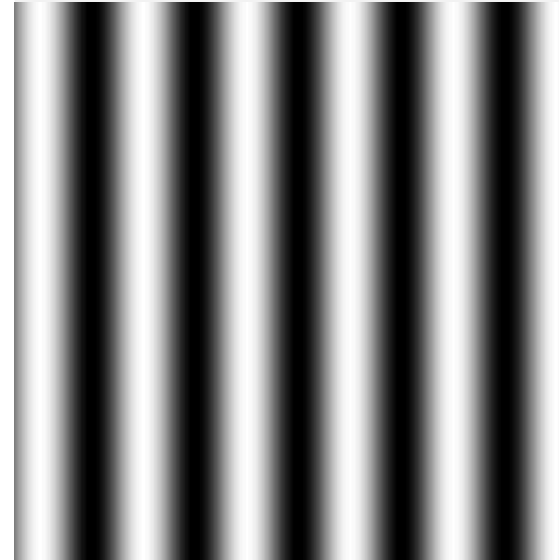
$$I(x, y) = x + y$$

Diagonal Gradient



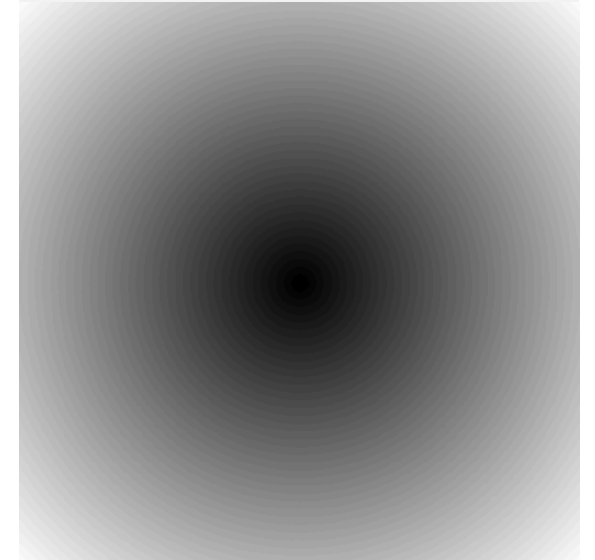
$$I(x, y) = \sin(x)$$

Periodical Pattern



$$I(x, y) = \sqrt{x^2 + y^2}$$

Radial Function



DISCRETE AND CONTINUOUS

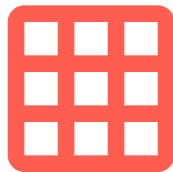
GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

IMAGE SYNTHESIS

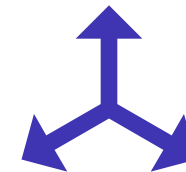
Discrete Space (Pixels)

- Coordinate system: Integers (0, 1, 2...)
- Fixed resolution (e.g., 1920x1080)
- Origin usually at Top-Left
- Atomic unit is the individual **Pixel Grid** cell



Continuous Space (Math)

- Coordinate system: Real Numbers (Floats)
- Infinite resolution (zoomable)
- **Normalized Coordinates** (0.0 to 1.0)
- Origin can be centered or custom



Mapping / Normalization

Converting pixel integers (0..W) to floats (0.0..1.0)

TRANSFORMATION

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

IMAGE SYNTHESIS

Input: Mathematical Range

$[-1.0, 1.0]$

Values from functions like $\sin()$ or noise



Output: Image Range

$[0, 255]$

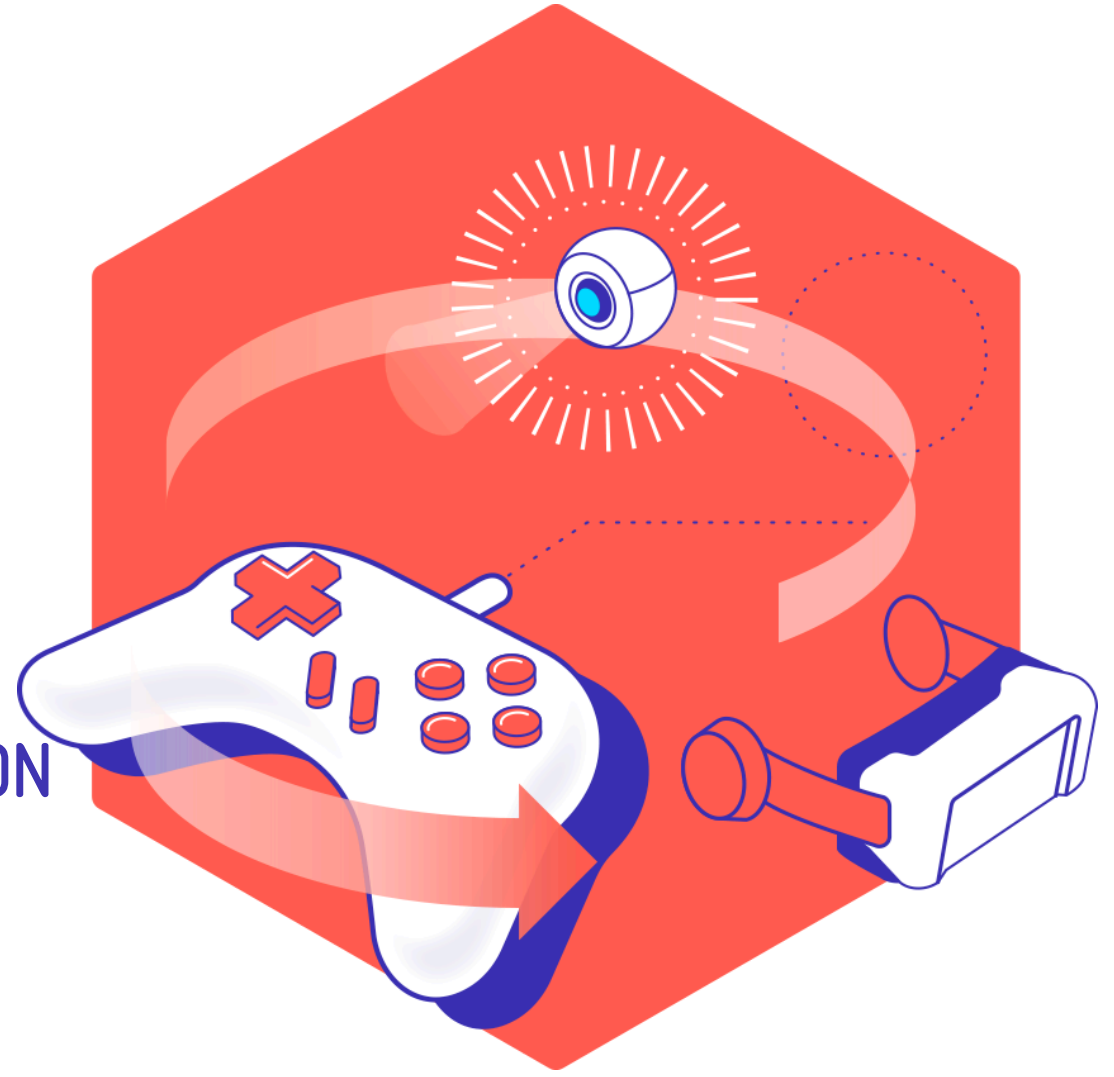
Standard 8-bit integer color values

The Transformation Process

- **Scaling:** Stretching the range (e.g., $\times 127.5$)
- **Mapping:** Shifting values (e.g., $+ 127.5$)
- **Clamping:** Restricting values to stay within a range (e.g., $[0, 255]$)
- **Clipping:** Discarding data outside a boundary

RANDOM NUMBERS

UNIT 4: ALGORITHMIC GRAPHICS GENERATION



WHY RANDOMNESS?

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

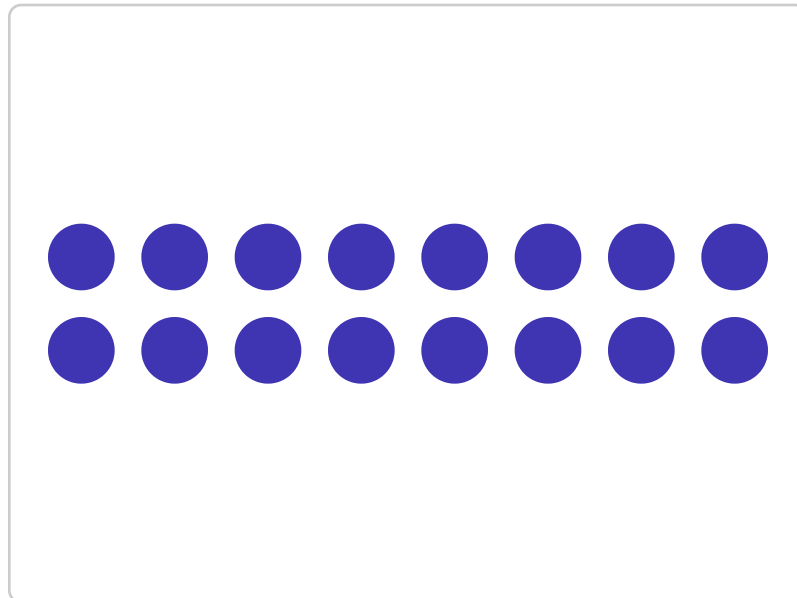
RANDOM NUMBERS

Perfection is Unnatural

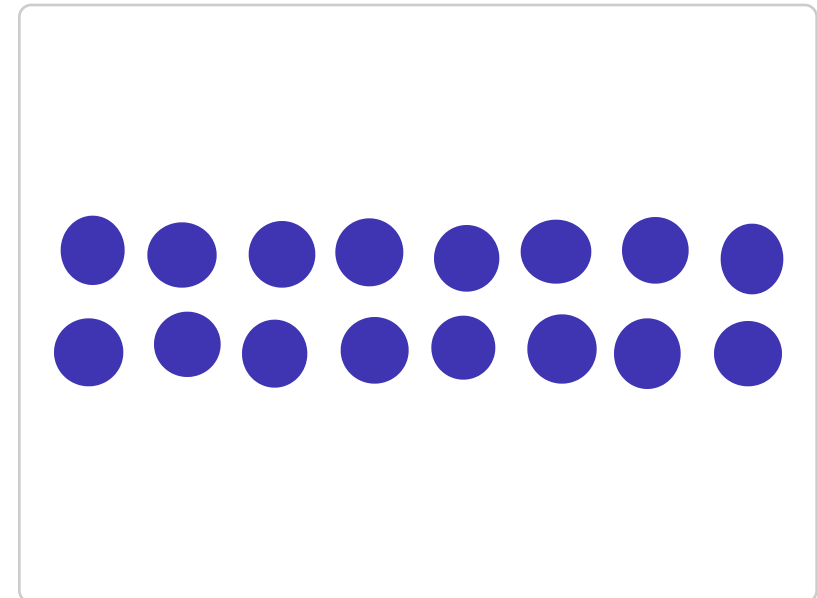
Natural structures rarely appear perfectly regular. Pure mathematical repetition looks "synthetic" and sterile.

Chance Creates Naturalness

Adding small random variations to size, color, or position breaks repetition and simulates the complexity of the real world.



Add Random Variation
→



PSEUDO-RANDOMNESS

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

RANDOM NUMBERS

Pseudo-Random Numbers (PRNG)

Computers are **deterministic** machines. They cannot generate "true" randomness without external hardware sensors.

- **Deterministic:** Given the same input (Seed), the algorithm always produces the same sequence.
- **Efficiency:** Fast to compute and reproducible.
- **Seed:** The starting value for the algorithm.

Deterministic Algorithms

PRNGs use mathematical formulas to create a sequence of numbers that *look* random.

RANDOM DISTRIBUTION

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

RANDOM NUMBERS

Uniform Distribution

Every value in the range has an equal probability of being chosen.

Used for:

- Base random generation
- Roll of a dice
- Selecting starting states



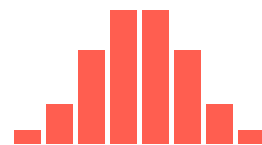
Equal Probability

Gaussian (Normal)

Values cluster around a central mean (the "Bell Curve").

Used for:

- Natural noise (Grain)
- Human traits (Height)
- Error distributions
- Motion blur jitter



Centered Mean

Stochastic Processes

Randomness that evolves over time or space (Random Walks).

Used for:

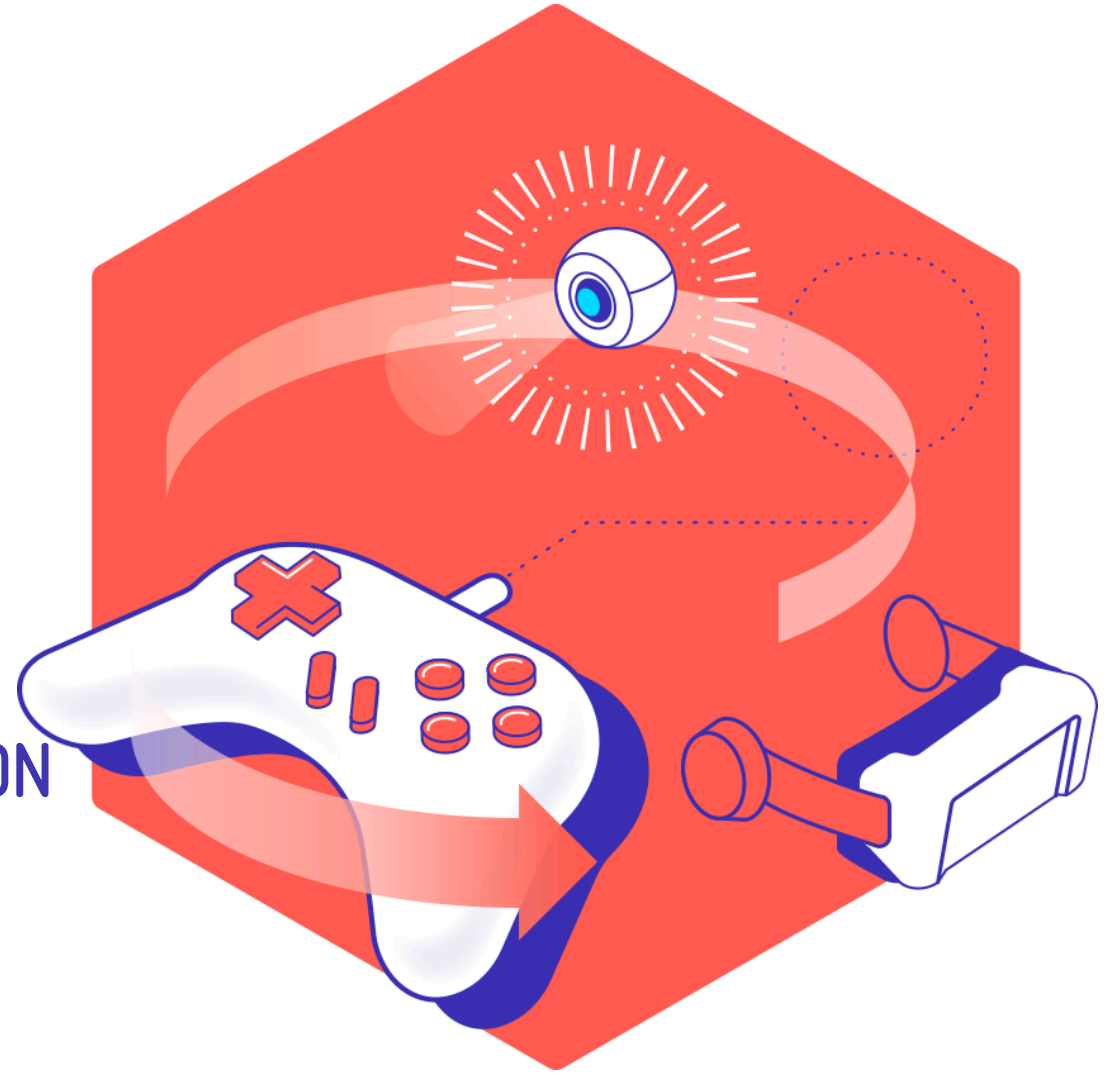
- Plant growth (L-systems)
- Smoke and fire particles
- Terrain erosion
- Stock market paths



State Memory

NOISE

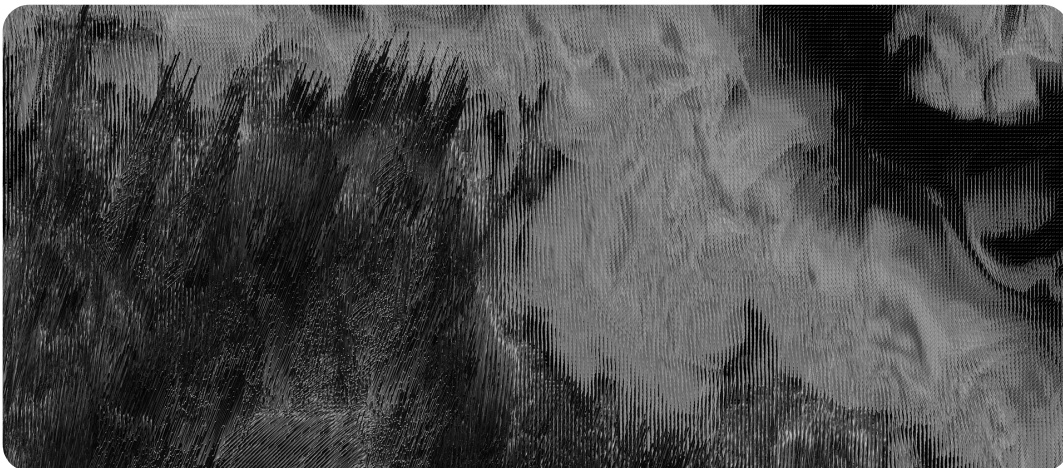
UNIT 4: ALGORITHMIC GRAPHICS GENERATION



WHAT IS NOISE?

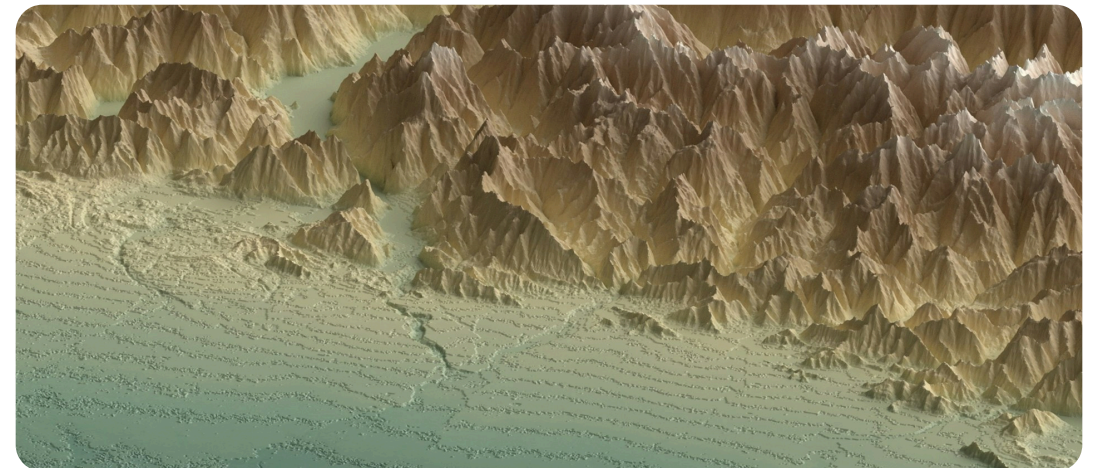
Definition

In graphics programming, **Noise** is a stochastic (random) signal used to simulate the complex, irregular patterns found in nature. Unlike pure random noise, graphical noise (like Perlin noise) is typically **smooth and continuous**.

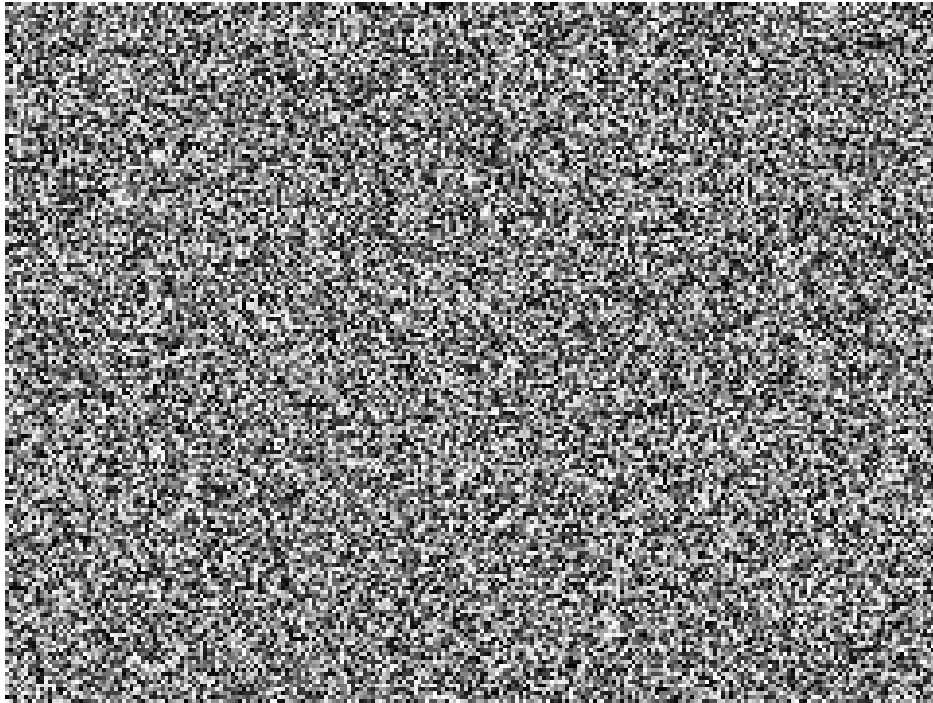


Common Examples

- **Procedural Terrain:** Heightmaps for mountains and valleys.
- **Organic Textures:** Wood grain, marble veins, and clouds.
- **VFX:** Particle motion for smoke, fire, and water.



WHITE NOISE



CC-BY-SA 3.0 Jorge Stolfi

THE PUREST RANDOMNESS

White noise is a signal with a constant power spectral density. In computer graphics, this means every pixel value is chosen completely independently of its neighbors.

Key Characteristics:

- **No Correlation:** No relationship between adjacent pixels.
- **Uniform Distribution:** Values are usually spread evenly across the $[0, 255]$ range.
- **Foundation:** Often the "seed" for more structured noise like Perlin or Simplex.

COHERENT NOISE

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

NOISE

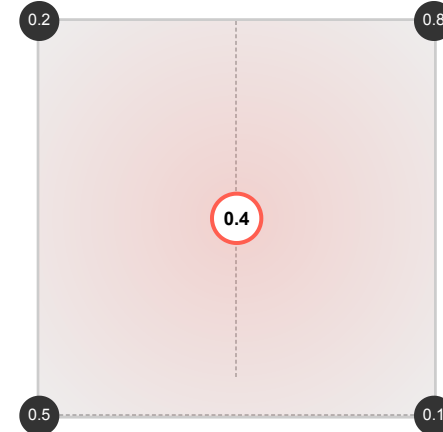
BEYOND WHITE NOISE

To create natural patterns, we need more than just independent random values. We need **Coherent Noise**: randomness where adjacent values are mathematically related.

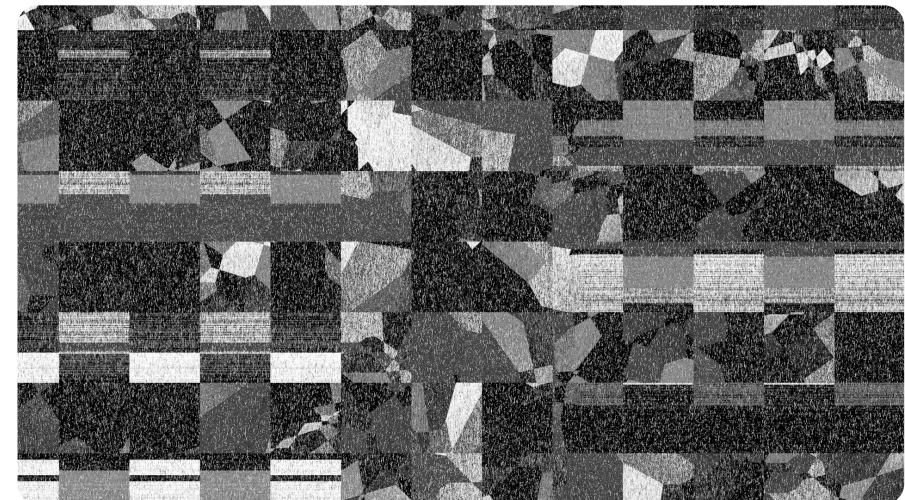
INTERPOLATION

The bridge between **random chaos** and **smooth structure**:

- Take random values at discrete points (the "grid").
- Use **interpolation** (linear, cosine, or cubic) to find values between them.
- The result is a continuous, smooth signal.

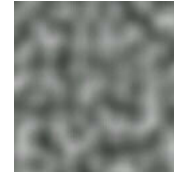


Values are assigned to grid intersections.
Bilinear interpolation calculates the value for any point inside the cell.



PERLIN NOISE

Developed by **Ken Perlin**, this "gradient noise" is the industry standard for procedural generation.



- **Continuous:** Smooth transitions between values.
- **Organic:** Mimics natural irregularities.
- **Efficient:** Mathematically predictable and scalable.

HISTORY & CULTURAL IMPACT

- **Tron (1982):** Ken Perlin developed the algorithm while working on this film at MAGI. He was frustrated by the "plastic" look of 80s CGI and wanted more natural textures.
- **Academy Award (1997):** Perlin received an Oscar (Technical Achievement Award) for the algorithm's massive impact on the film industry.
- **Minecraft:** The blocky, infinite terrain generation relies heavily on 2D and 3D Perlin/Simplex noise to determine biome heightmaps and cave structures.

PERLIN NOISE 2D

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

NOISE

1

GRID DEFINITION

Define an n -dimensional grid. For each grid intersection, assign a **random gradient vector**.

2

DISTANCE VECTORS

For a point (x,y) , calculate four **distance vectors** pointing from the cell corners to the point.

3

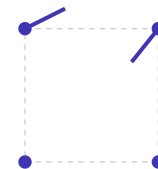
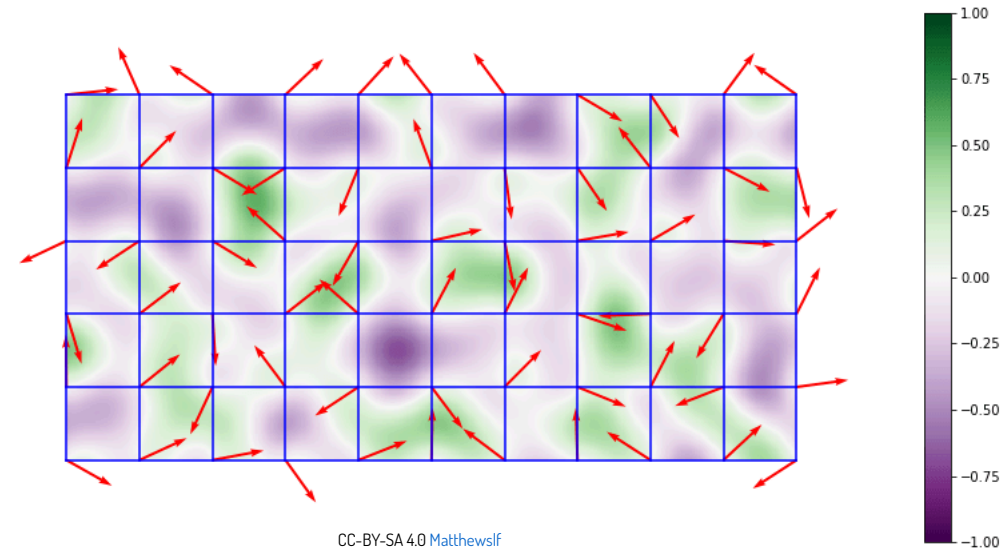
DOT PRODUCTS

Compute the **dot product** between the gradient vector and the distance vector at each corner.

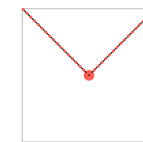
4

INTERPOLATION

Linearly interpolate the four dot products using a **fade function** for smooth transitions.



Gradients



Distances



Smoothstep

Interpolation

OCTAVES / FRACTAL NOISE

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

NOISE

FRACTAL NOISE (OCTAVES)

Perlin noise on its own is often too "smooth". To create complex, natural textures, we combine multiple layers of noise together.

The Components

- **Octave:** Each individual layer of noise.
- **Frequency:** Grid divisions. Higher frequency = smaller features.
- **Amplitude:** The "height" or strength.

Why Use It?

Mimics **Self-Similarity** found in nature (mountains, clouds, coastlines). Each octave adds a new level of detail, from landmasses to pebbles.

$$\text{Result} = \sum (\text{Octave}_n * \text{Amp}_n)$$

As frequency doubles, amplitude is halved (Persistence).

Octave 1
(freq=1, amp=1)



Octave 2
(freq=2, amp=0.5)



Octave 3
(freq=4, amp=0.25)

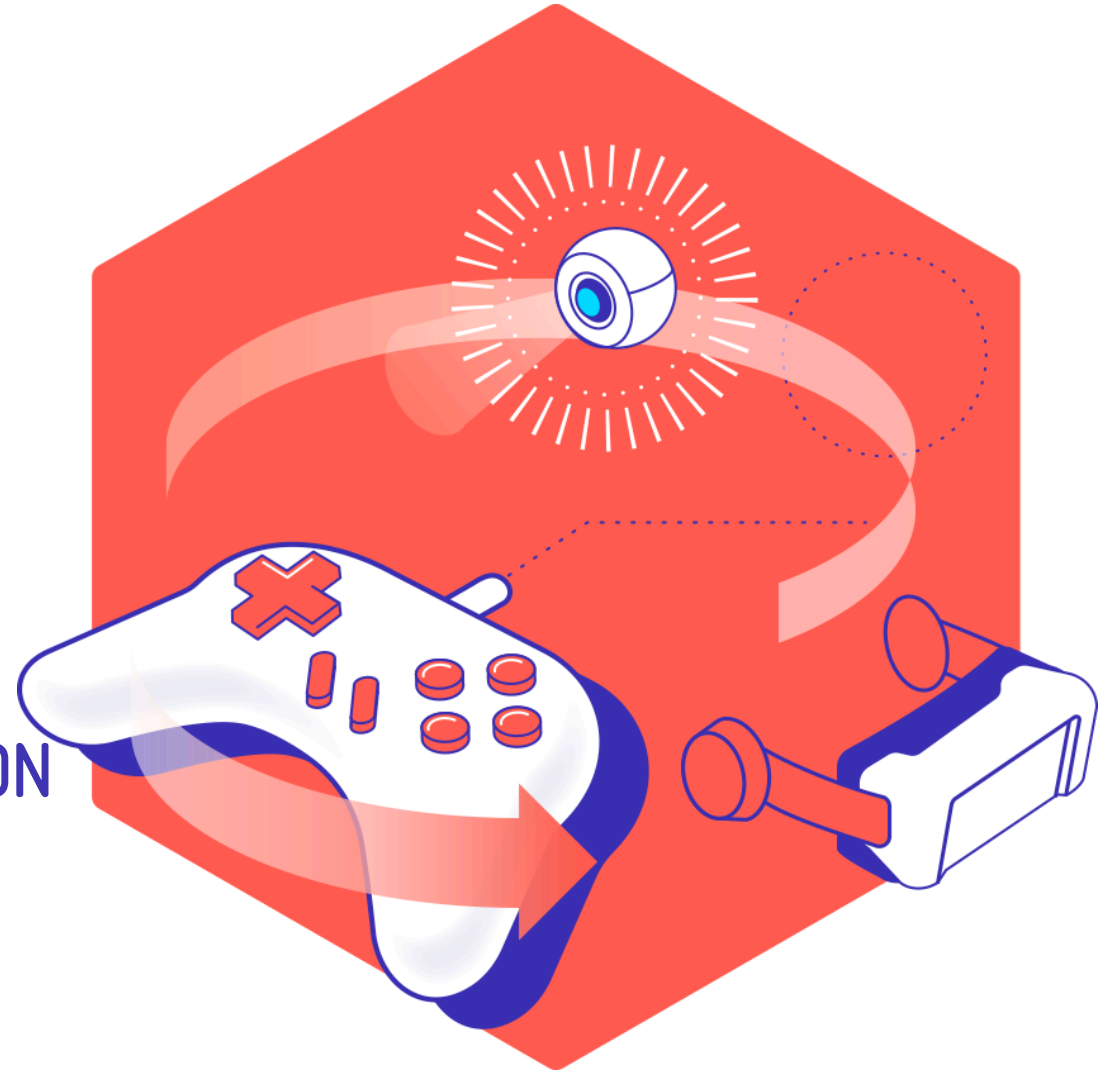


Overlay
(fractal sum)



FRACTALS

UNIT 4: ALGORITHMIC GRAPHICS GENERATION



SELF-SIMILARITY

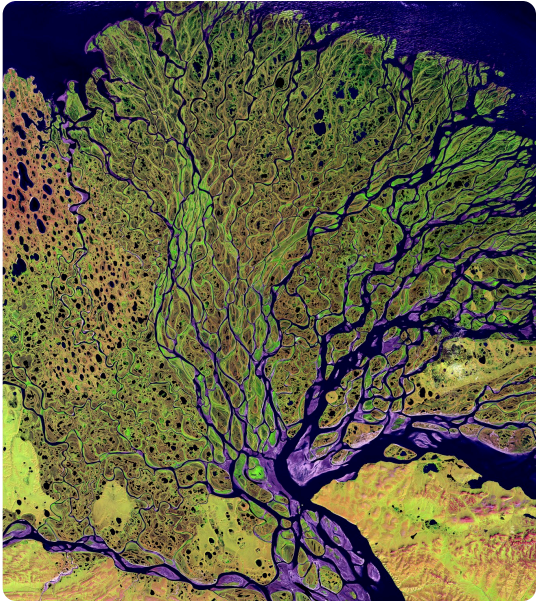
GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

FRACTALS

DEFINITION

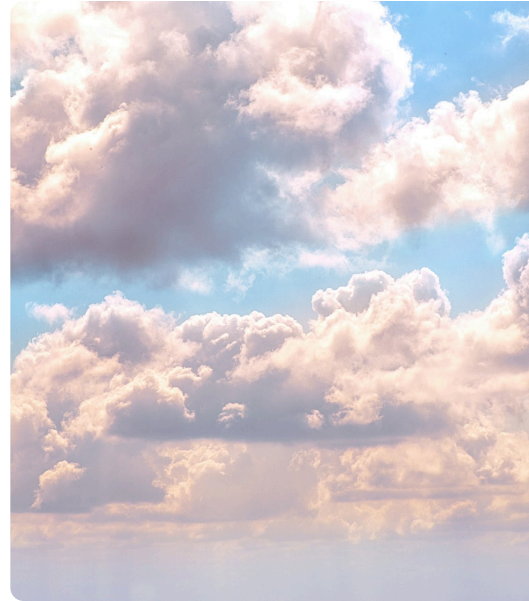
Structures that appear **similar across different scales** of observation. Zooming in reveals patterns that resemble the whole.



Coastlines



Plants (Romanesco)



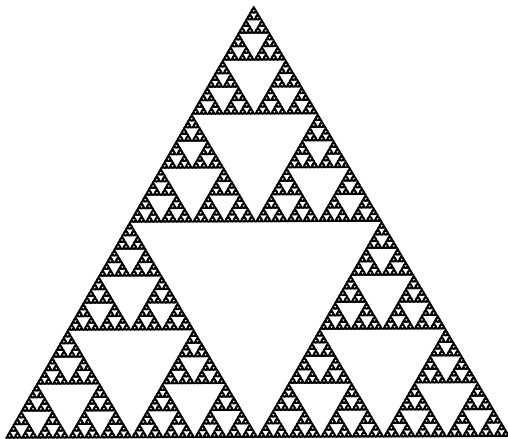
Clouds

FRACTALS

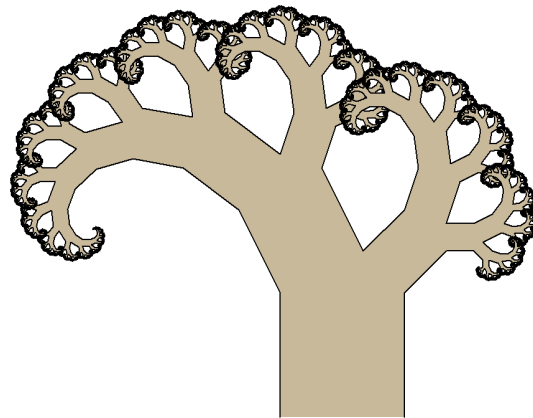
DEFINITION

A **fractal** is a complex geometric shape that exhibits **self-similarity** across different scales. This means if you zoom in on any part of the fractal, you find a smaller version that resembles the whole.

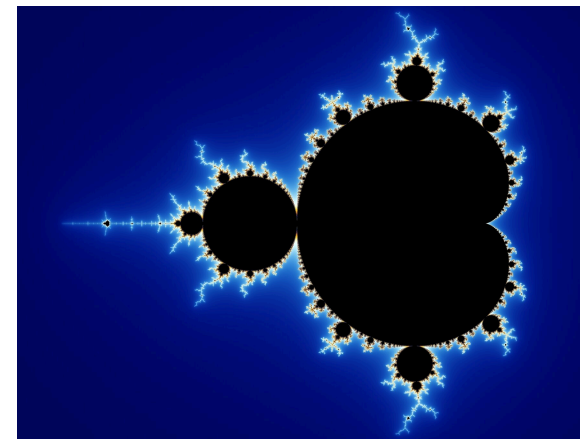
- **Infinite Complexity:** Patterns repeat infinitely.
- **Fractional Dimension:** They exist between standard Euclidean dimensions (1D, 2D, 3D).



Sierpinski Triangle



Pythagoras Tree



Mandelbrot Set

KOCH CURVE

GRAPHICS PROGRAMMING

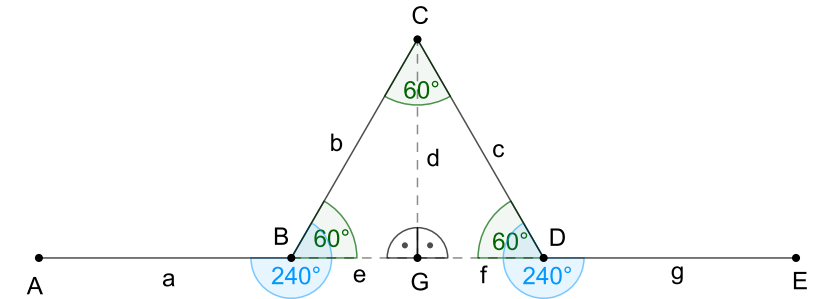
UNIT 4: ALGORITHMIC GRAPHICS GEN.

FRACTALS

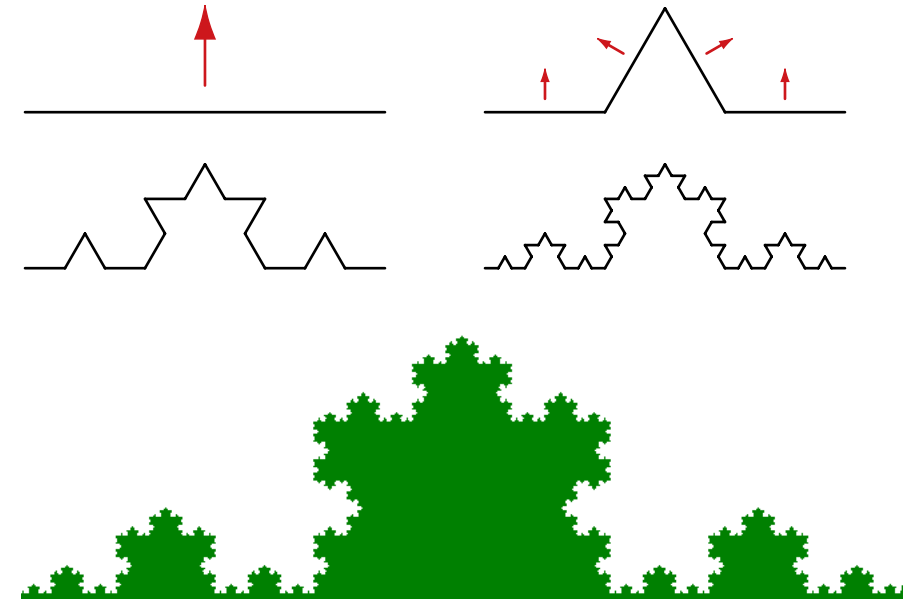
HOW TO BUILD IT

1. **The Base:** Start with a single straight line segment.
2. **Divide:** Split the line into **three equal parts**.
3. **The Peak:** Remove the middle part and replace it with two sides of an equilateral triangle.
4. **Repeat:** Now you have four shorter lines. Apply these same steps to **each** of them.

This recursive process creates an infinitely long line that fits within a finite area!



CC BY 4.0 Kmhkmh



MANDELBROT SET

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

FRACTALS

Generated by the simple formula:

$$z_{n+1} = z_n^2 + c$$

Where c and z are complex numbers.

HOW IT WORKS

1. Complex Numbers as Coordinates

Think of each pixel on your screen as a coordinate c on a 2D plane. The X-axis represents real numbers, and the Y-axis represents imaginary numbers.

2. The Iteration Loop

For every pixel, we run a recursive calculation repeatedly (starting at $z=0$). We watch what happens to the value:

- **Stays small:** It's inside the set (colored black).
- **Explodes to infinity:** It's outside the set.

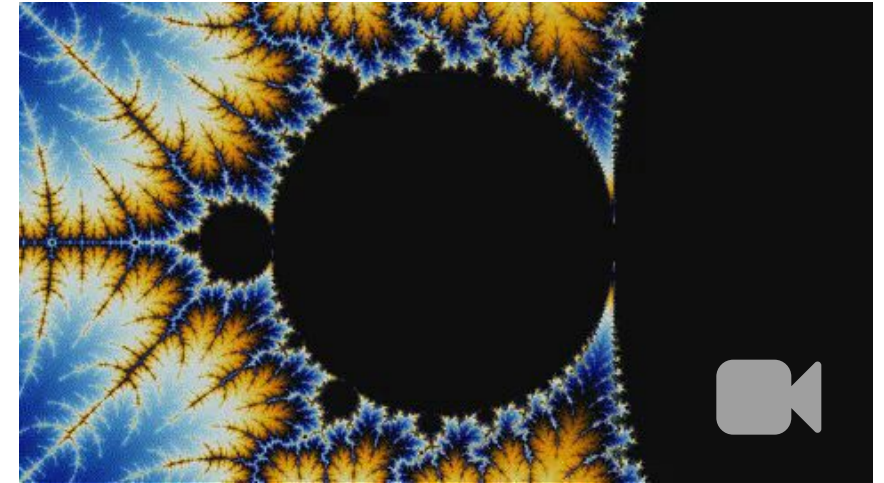
3. Visualization

To get those beautiful colors, we count *how many iterations* it took for the number to "escape." Different speeds get different colors!

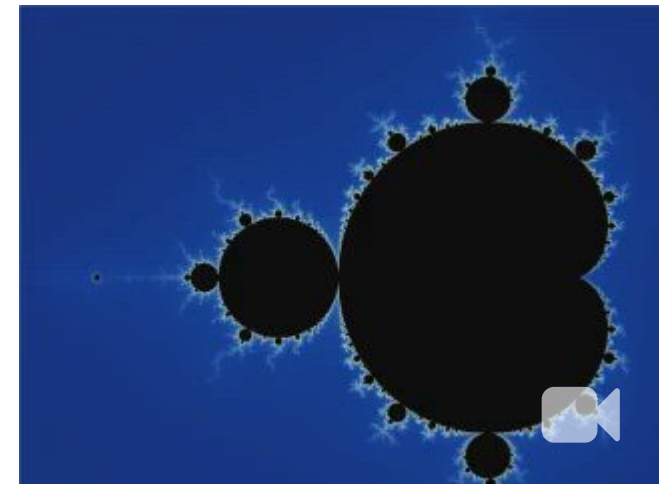
STEP-BY-STEP ALGORITHM

for each Pixel:

1. **Pick a Point:** Determine c
2. **Initialize:** Set your starting value $z = 0$.
3. **Loop (Iterate):** Keep updating z using:
$$z_{n+1} = z_n^2 + c$$
4. **Check:** Does the magnitude of z exceed 2? ($|z_n| > 2$)
 - **Yes:** Stop! The point escaped. Color it based on how many loops it took.
 - **No:** Repeat step 3 until you reach a maximum limit (e.g. $\text{max_iterations} = 100$)
5. **Final Result:** If it never escaped, color the pixel black.



CC BY-SA 4.0 Danielkwalsh



MANDELBROT SET

GRAPHICS PROGRAMMING

UNIT 4: ALGORITHMIC GRAPHICS GEN.

FRACTALS

MAPPING PIXELS TO "C"

To render the set, we must translate screen coordinates (pixels) into the complex plane coordinates used in the formula.

1. Identify Screen Coordinates:

Take the pixel's position (x, y) and the total screen size $(width, height)$.

2. Normalize (Scale to 0-1):

Calculate the percentage of the way across the screen:

$$\text{norm_x} = x / \text{width}$$

$$\text{norm_y} = y / \text{height}$$

3. Map to Complex Plane:

Map these 0-1 values to the desired fractal range (usually -2 to 1 for Real, -1 to 1 for Imaginary):

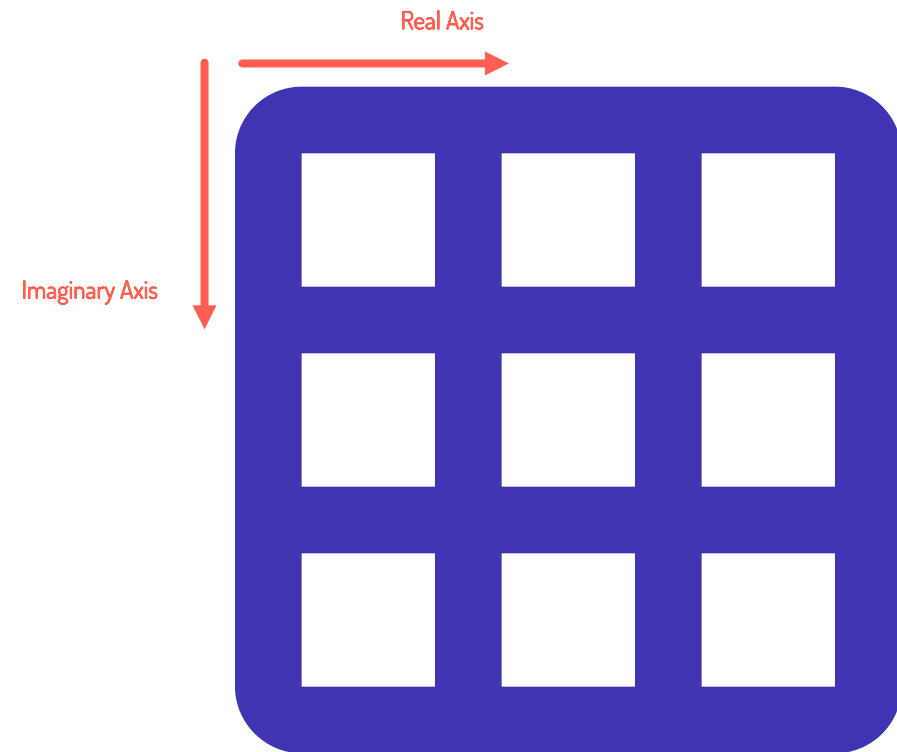
$$\text{Real}(c) = \text{xmin} + (\text{norm_x} * \text{range_re})$$

$$\text{Imag}(c) = \text{ymin} + (\text{norm_y} * \text{range_im})$$

4. Form the Number:

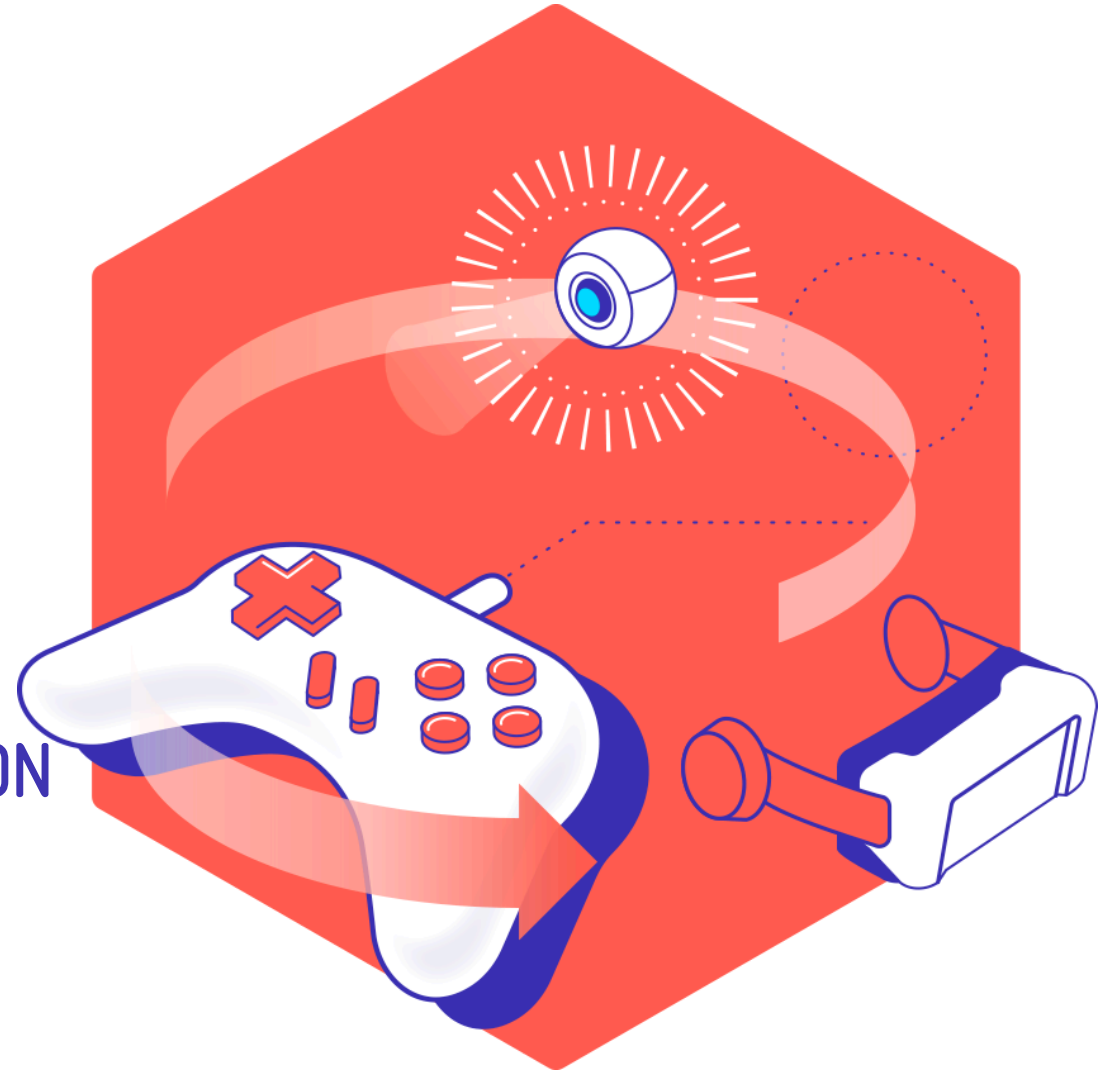
The complex number is now $c = \text{Real}(c) + \text{Imag}(c)i$.

```
1 real = xmin + (x / width) * (xmax - xmin)
2
3 imag = ymin + (y / height) * (ymax - ymin)
4
5 c = complex(real, imag)
```



END

UNIT 4: ALGORITHMIC GRAPHICS GENERATION



Unless otherwise noted, the contents of this Open Educational Resource are licensed under the Creative-Commons Attribution 4.0 International license (CC BY 4.0)
Attribution: Eike Langbehn, Peter Wood

